

Performance evaluation of parallel programs

Pacheco, Section 2.6

Moreno Marzolla
Dip. di Informatica—Scienza e Ingegneria (DISI)
Università di Bologna

Copyright © 2013, 2014, 2017–2024
Moreno Marzolla, Università di Bologna, Italy
<https://www.moreno.marzolla.name/teaching/HPC/>



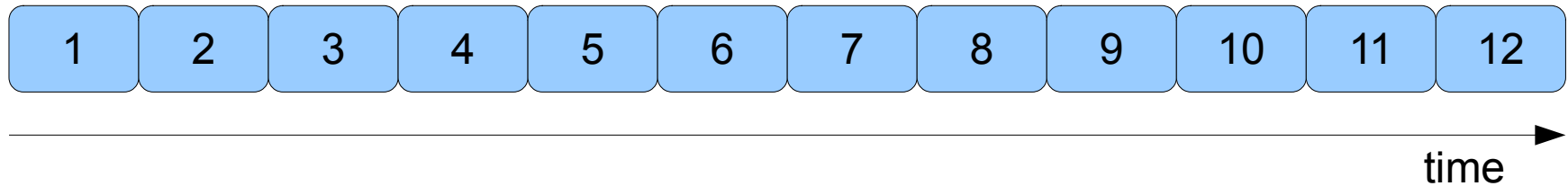
Except where stated otherwise, this work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC-BY-SA 4.0). To view a copy of this license, visit <https://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

Credits

- prof. David Padua
 - <https://courses.engr.illinois.edu/cs420/fa2012/lectures.html>
- Prof. Guy Blelloch

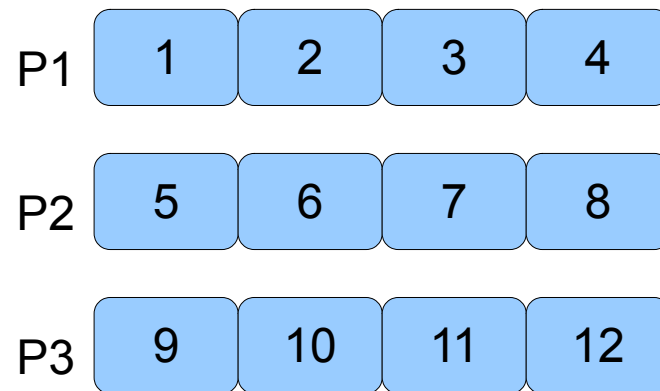
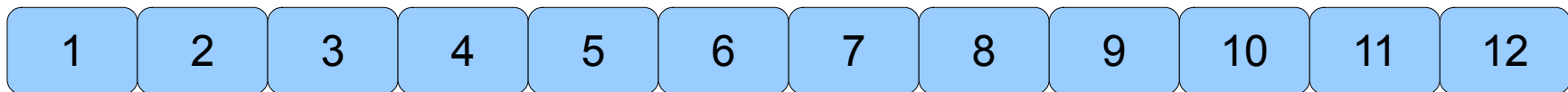
How fast can we run?

- 12 tasks, each one requiring 1s
- Total serial time: 12s



How fast can we run?

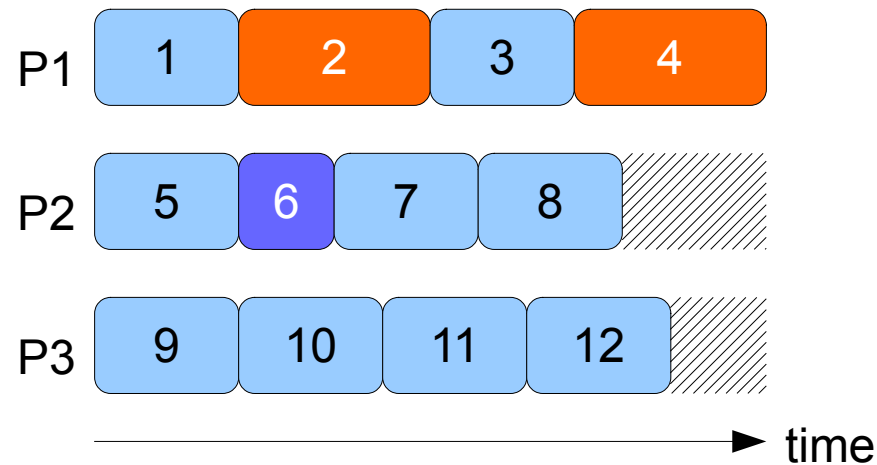
- What if we have 3 processors and all tasks are independent?
 - Execution time becomes 4s



Performance Evaluation → time

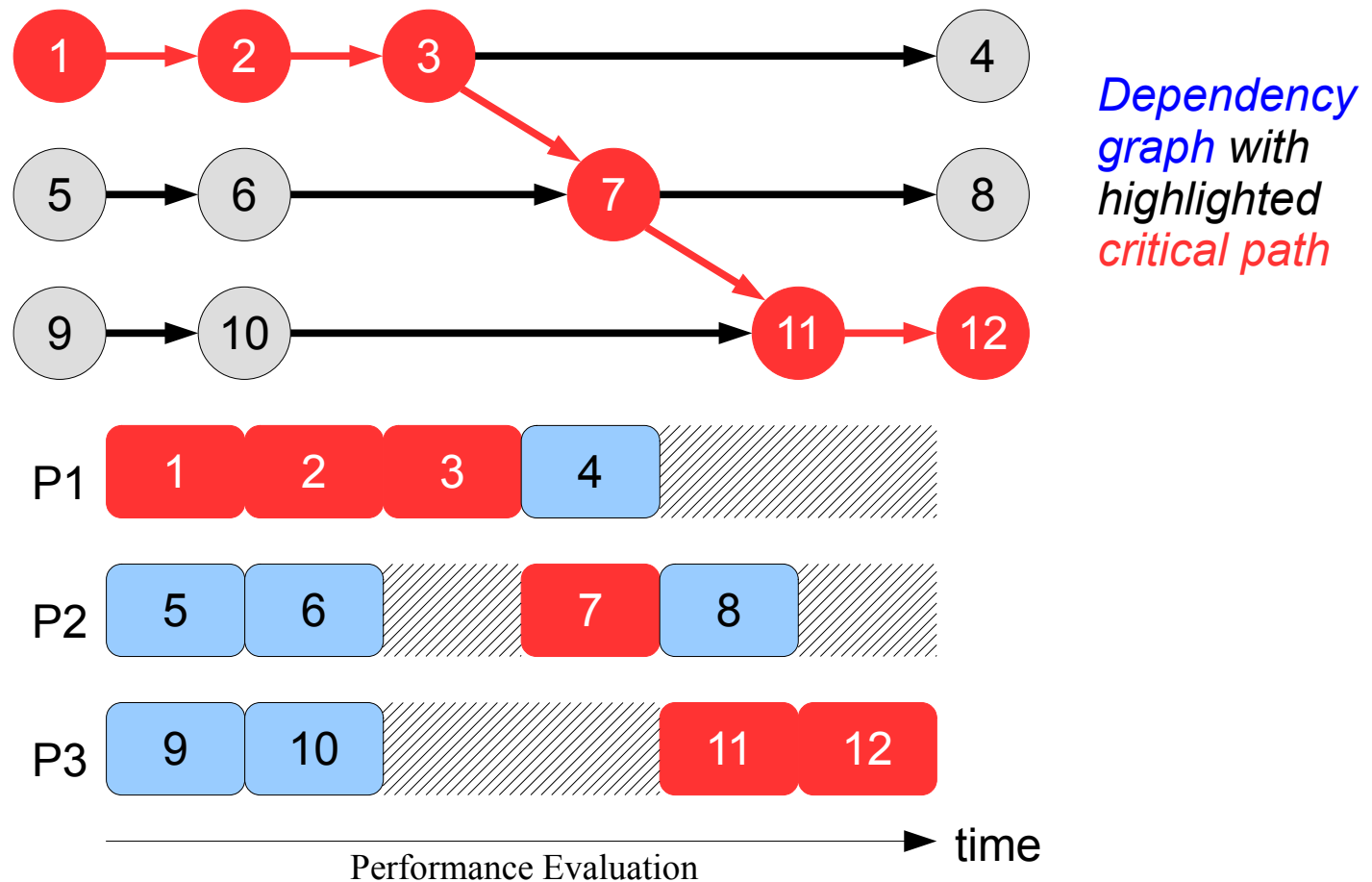
How fast can we run?

- What if the processors can not execute the tasks with the same speed?
 - **Load imbalance** (ending part of P2 and P3)



How fast can we run?

- What if tasks are dependent?
 - Execution time grows from 4s to 6s



Scalability

- How much faster can a given problem be solved with p workers instead of one?
- How much more work can be done with p workers instead of one?
- What impact for the communication requirements of the parallel application have on performance?
- What fraction of the resources is actually used productively for solving the problem?

Speedup

- Let us define:
 - p = Number of processors / cores
 - T_{serial} = Execution time of the serial program
 - $T_{\text{parallel}}(p)$ = Execution time of the parallel program with p processors / cores

Speedup

- Speedup $S(p)$

$$S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)} \approx \frac{T_{\text{parallel}}(1)}{T_{\text{parallel}}(p)}$$

- In the ideal case, the parallel program requires $1/p$ the time of the sequential program
- $S(p) = p$ is the ideal case of optimal speedup
 - Realistically, $S(p) \leq p$
 - Is it possible to observe $S(p) > p$?



Warning



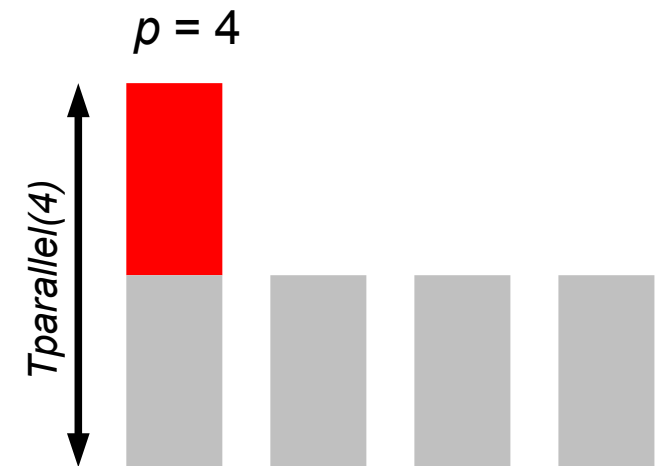
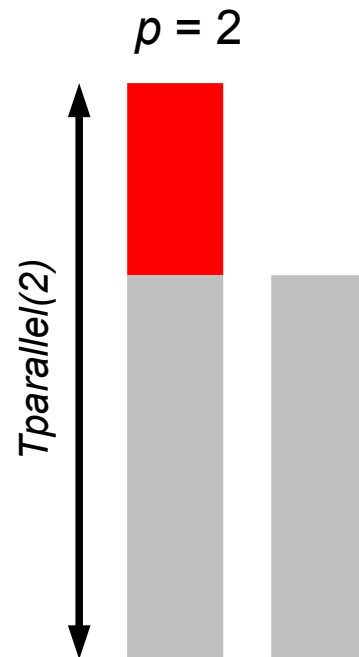
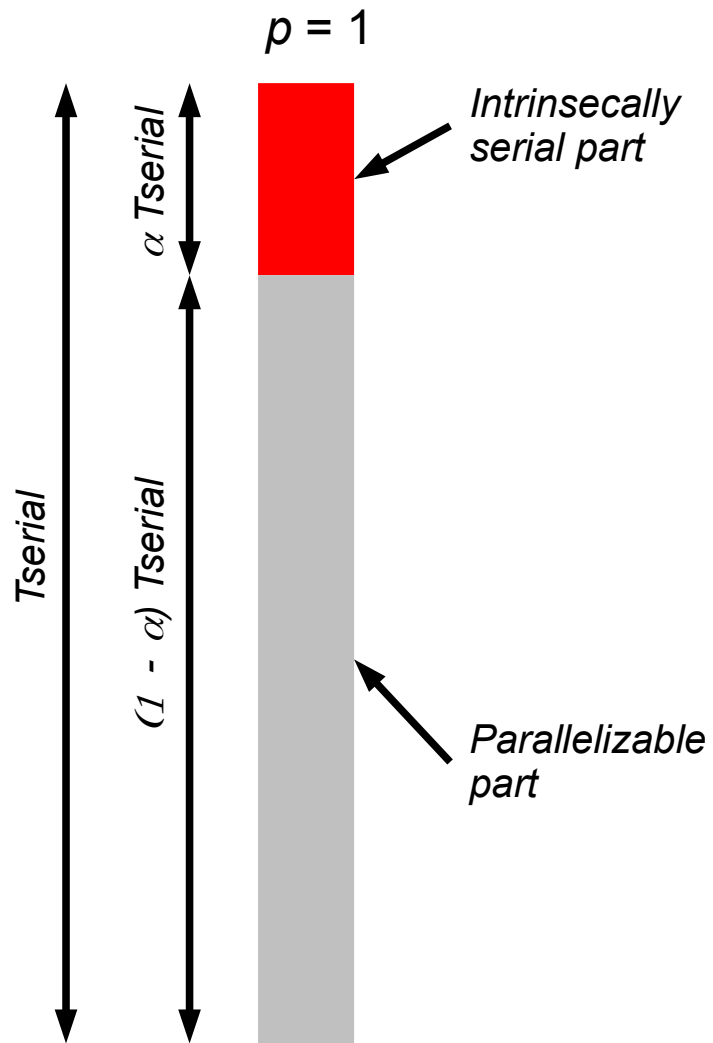
- Do not use a serial program to compute T_{serial}
 - If you do that, you might see a spurious superlinear speedup that is not there
- Always use the parallel program with $p = 1$ processors

Non-parallelizable portions

- Suppose that a fraction α of the total execution time of the serial program can **not** be parallelized
 - E.g., due to:
 - Algorithmic limitations (data dependencies)
 - Bottlenecks (e.g., shared resources)
 - Startup overhead
 - Communication costs
- Suppose that the remaining fraction $(1 - \alpha)$ can be **fully parallelized**
- Then, we have:

$$T_{\text{parallel}}(p) = \alpha T_{\text{serial}} + \frac{(1 - \alpha) T_{\text{serial}}}{p}$$

Example



$$T_{\text{parallel}}(p) = \alpha T_{\text{serial}} + \frac{(1 - \alpha) T_{\text{serial}}}{p}$$

Example

- Suppose that a program has $T_{\text{serial}} = 20\text{s}$
- Assume that 10% of the time is spent in a serial portion of the program
- Therefore, the execution time of a parallel version with p processors is

$$T_{\text{parallel}}(p) = 0.1 T_{\text{serial}} + \frac{0.9 T_{\text{serial}}}{p} = 2 + \frac{18}{p}$$

Example (cont.)

- The speedup is

$$S(p) = \frac{T_{\text{serial}}}{0.1 \times T_{\text{serial}} + \frac{0.9 \times T_{\text{serial}}}{p}} = \frac{20}{2 + \frac{18}{p}}$$

- What is the maximum speedup that can be achieved when $p \rightarrow +\infty$?

Amdahl's Law

- What is the maximum speedup?

$$\begin{aligned} S(p) &= \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)} \\ &= \frac{T_{\text{serial}}}{\alpha T_{\text{serial}} + \frac{(1-\alpha) T_{\text{serial}}}{p}} \\ &= \frac{1}{\alpha + \frac{1-\alpha}{p}} \end{aligned}$$



Gene Myron Amdahl (1922—)

Amdahl's Law

- From

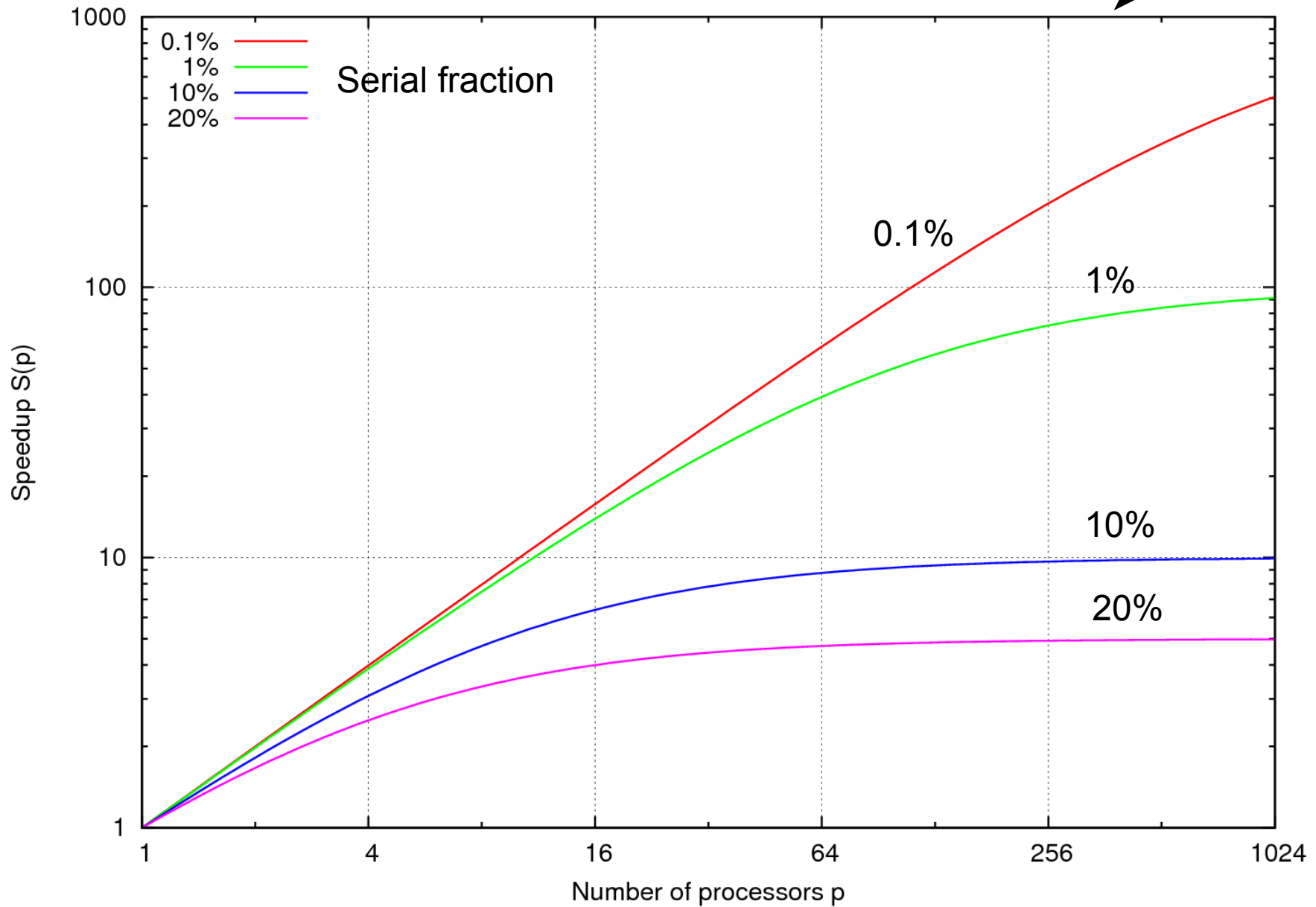
$$S(p) = \frac{1}{\alpha + \frac{(1-\alpha)}{p}}$$

we get an asymptotic speedup $1 / \alpha$ when p grows to infinity

If a fraction α of the total execution time is spent on a serial portion of the program, then the maximum achievable speedup is $1/\alpha$

Speedup

Note: log-log scale



Scaling Efficiency

- **Strong Scaling**: increase the number of processors p keeping the *total* problem size fixed
 - The *total* amount of work remains constant
 - The amount of work for a single processor decreases as p increases
 - Goal: reduce the total execution time by adding more processors
- **Weak Scaling**: increase the number of processors p keeping the *per-processor* work fixed
 - The total amount of work grows as p increases
 - Goal: solve larger problems within the same amount of time

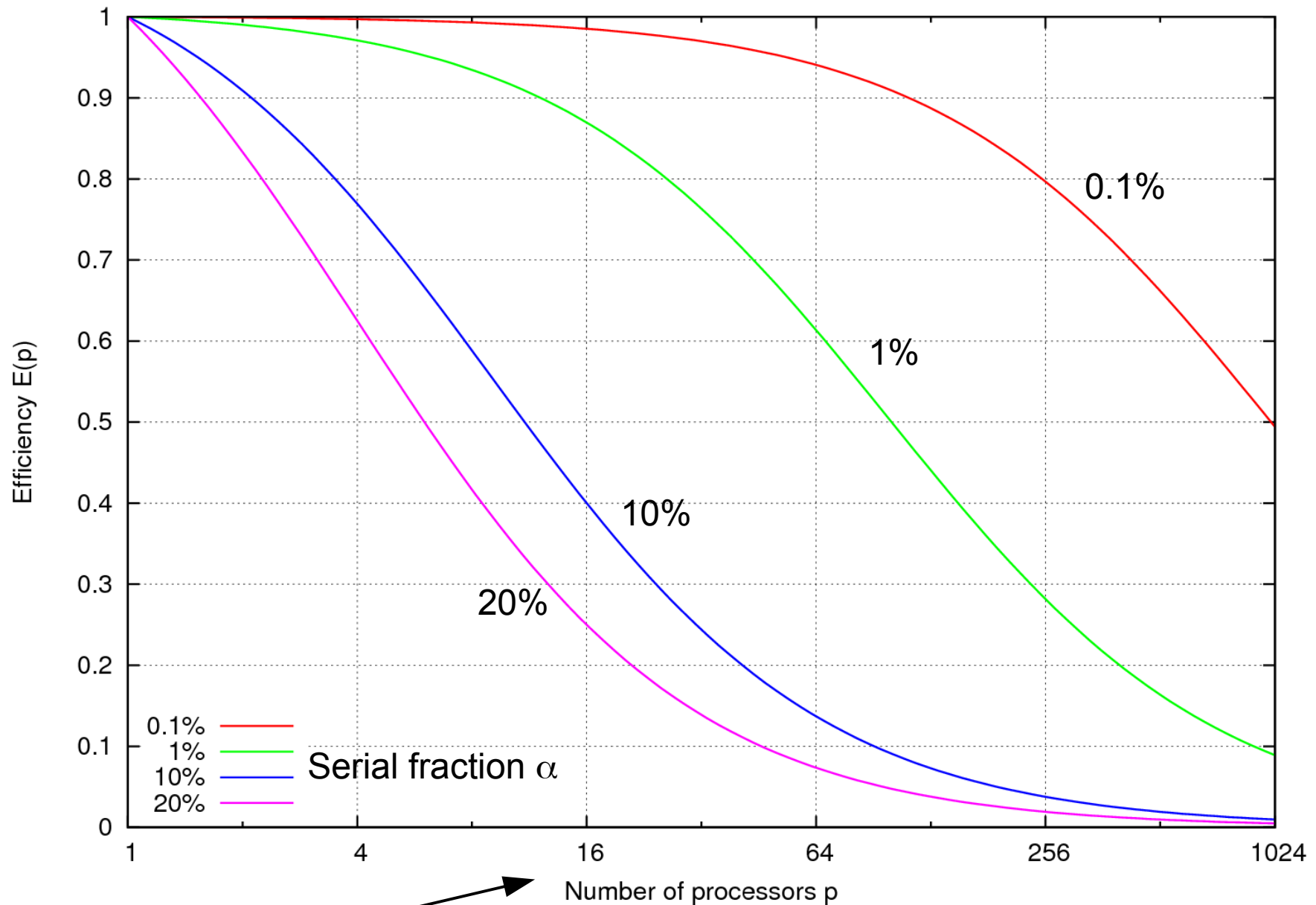
Strong Scaling Efficiency

- $E(p)$ = Strong Scaling Efficiency

$$E(p) = \frac{S(p)}{p} = \frac{T_{\text{parallel}}(1)}{p \times T_{\text{parallel}}(p)}$$

- where
 - $T_{\text{parallel}}(p)$ = Execution time of the parallel program with p processors / cores

Strong Scaling Efficiency and Amdahl's Law



Note: logarithmic x scale

Weak Scaling Efficiency

- $W(p)$ = Weak Scaling Efficiency

$$W(p) = \frac{T_1}{T_p}$$

- where
 - T_1 = time required to complete 1 work unit with 1 processor
 - T_p = time required to complete p work units with p processors

Weak scaling

- Let $f(n_p, p)$ be the amount of work done by each processor, given input size n_p and p processors
- We want to compute the input size(s) n_p such that:

$$f(n_p, p) = \text{constant}$$

Example

Matrix-Matrix multiply

- For a given n_p , the amount of serial “work” required to compute the product of two $n_p \times n_p$ matrices is $O(n_p^3)$
- The OpenMP version using p processors performs $f(n_p, p) = n_p^3 / p$ work per OpenMP thread
- Therefore:

$$\begin{aligned} n_p^3 / p &= \text{const} \\ n_p &= \sqrt[3]{p \times \text{const}} \\ n_p &= \sqrt[3]{p} \times \text{const}' \end{aligned}$$

Example

- See [omp-matmul.c](#)
 - [demo-strong-scaling.sh](#) computes the times required for strong scaling (demo-strong-scaling.ods)
 - [demo-weak-scaling.sh](#) computes the times required for weak scaling (demo-weak-scaling.ods)

Taking times

- To compute speedup and efficiencies one needs to take the program **wall clock time**, excluding the input/output time

```
#if _XOPEN_SOURCE < 600  
#define _XOPEN_SOURCE 600  
#endif
```

```
#include "hpc.h"
```

```
other #includes here...
```

```
...
```

```
double start, finish;
```

```
/* read input data; this should not be measured */
```

```
start = hpc_gettime();
```

```
/* actual code that we want to time */
```

```
finish = hpc_gettime();
```

```
printf("Elapsed time = %.3f seconds\n", finish - start);
```

```
/* write output data; this should not be measured */
```

Must be at the very beginning
(can be omitted for CUDA)

The actual
computation is done
here

How to measure the wall-clock time

- Using OpenMP: `omp_get_wtime()`
- Using MPI: `MPI_Wtime()`
- Generic solution: `clock_gettime()`
- **Wrong solution: `clock()`**

NAME

`clock` - Determine processor time

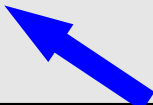
SYNOPSIS

```
#include <time.h>
```

```
clock_t clock(void);
```

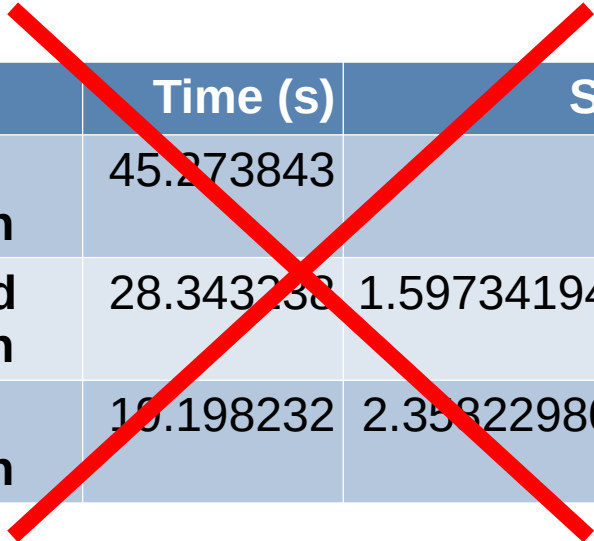
DESCRIPTION

The `clock()` function returns an approximation of **processor time** used by the program.



Displaying times

- When displaying times in seconds, e.g. in data tables, **do not show too many decimal digits**
 - Rule of thumb: at most 3 decimal places (milliseconds)
 - Interference from other processes, the OS, I/O, ... introduce errors that are orders of magnitude higher



	Time (s)	Speedup
First version	45.273843	1
Second version	28.343238	1.59734194801596
Third version	19.198232	2.35822980991166

	Time (s)	Speedup
First version	45.274	1.00
Second version	28.343	1.60
Third version	19.198	2.36

Important

When showing performance results, always provide the **hardware specification**.

For the CPU

- Type of processor (vendor, model name, ecc.)
- Number of cores
- Whether the CPU uses symmetric multithreading (e.g., HyperThreading)
- Clock frequency
- Amount of RAM
- Operating System
- Compiler version and possibly compilation flags

For the GPU

- Type of GPU (vendor, model name, ecc.)
- Number of cores
- Clock frequency
- Amount of device memory
- Compiler version and possibly compilation flags

Post Scriptum: How to present results

Basic rules for graphing data

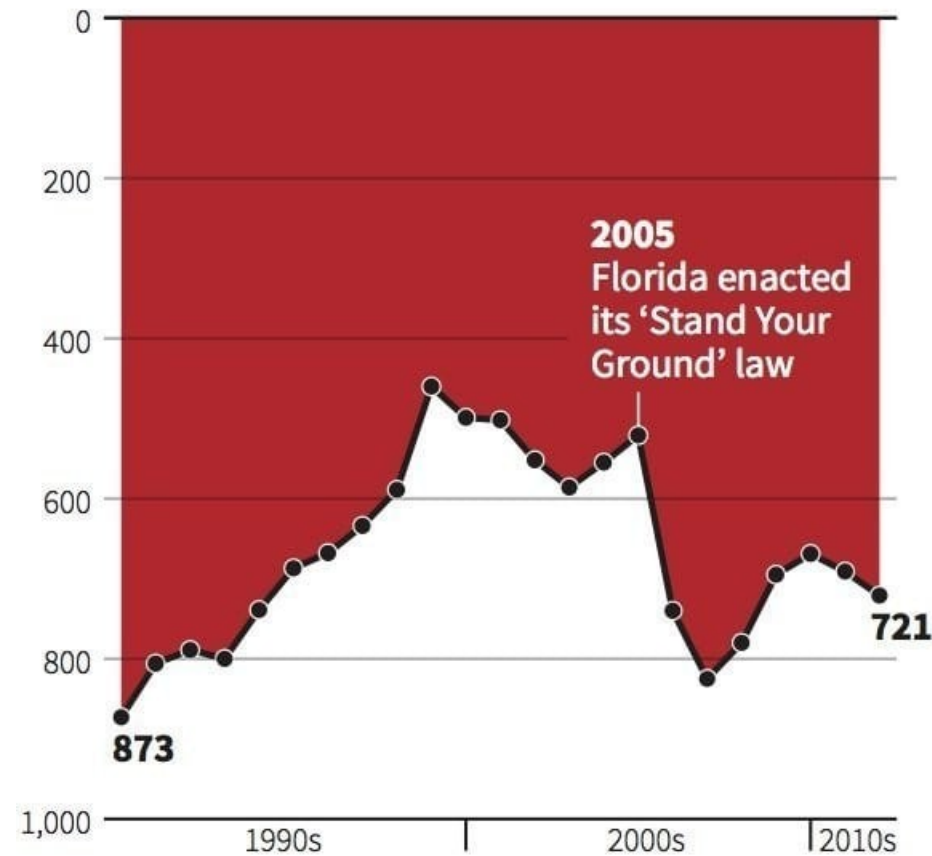
- Understanding the data should require the least possible effort from the reader
 - E.g., use names instead of symbols for the legend when possible
- Provide enough information to make the graph self-contained
- Stick to widely accepted practices
 - The origin of plots should be (0, 0)
 - The effect should be on the y axis, the cause on the x axis

Stick to accepted practices

Bad

Gun deaths in Florida

Number of murders committed using firearms



Source: Florida Department of Law Enforcement

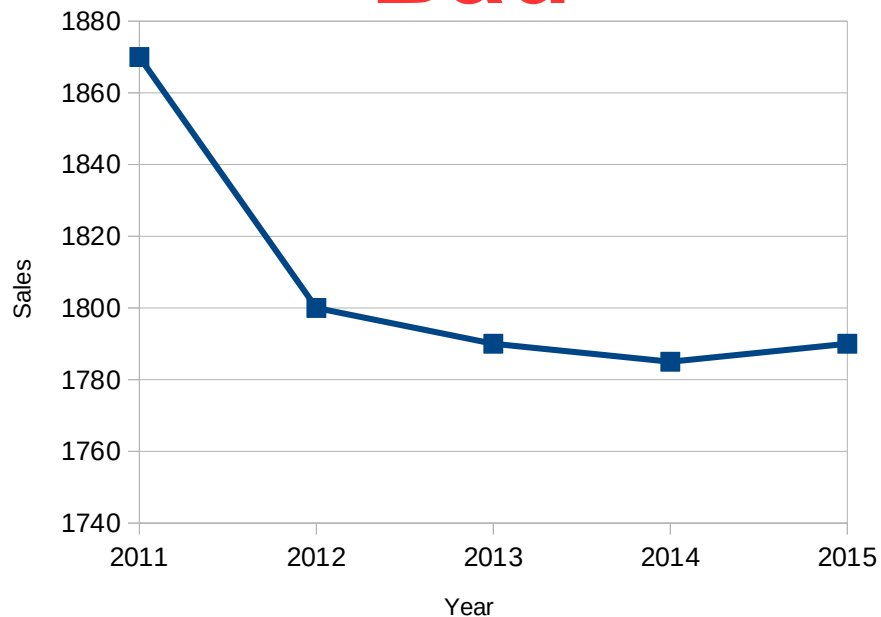
C. Chan 16/02/2014

REUTERS

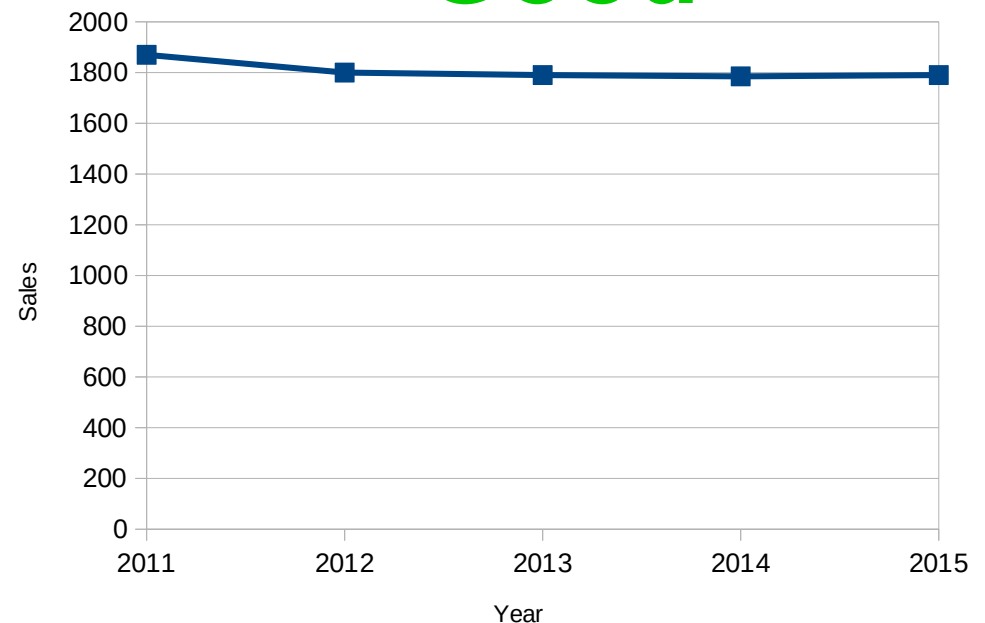
Do not misuse axis origins

- Playing with the axis ranges allows one to emphasize small differences

Bad



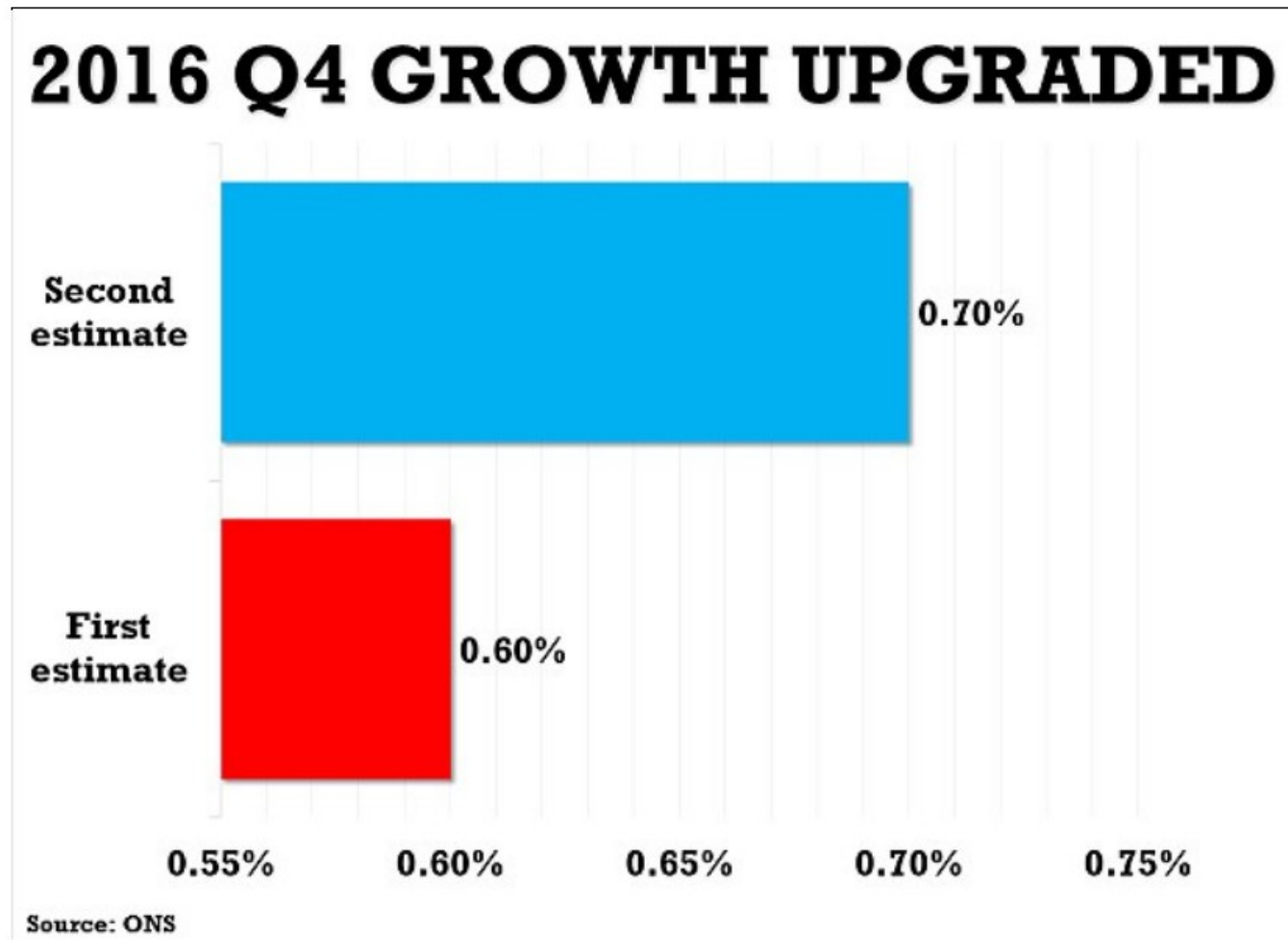
Good



Do not misuse axis origins

a change from 0.6% to 0.7% according to the Daily Mail

Bad



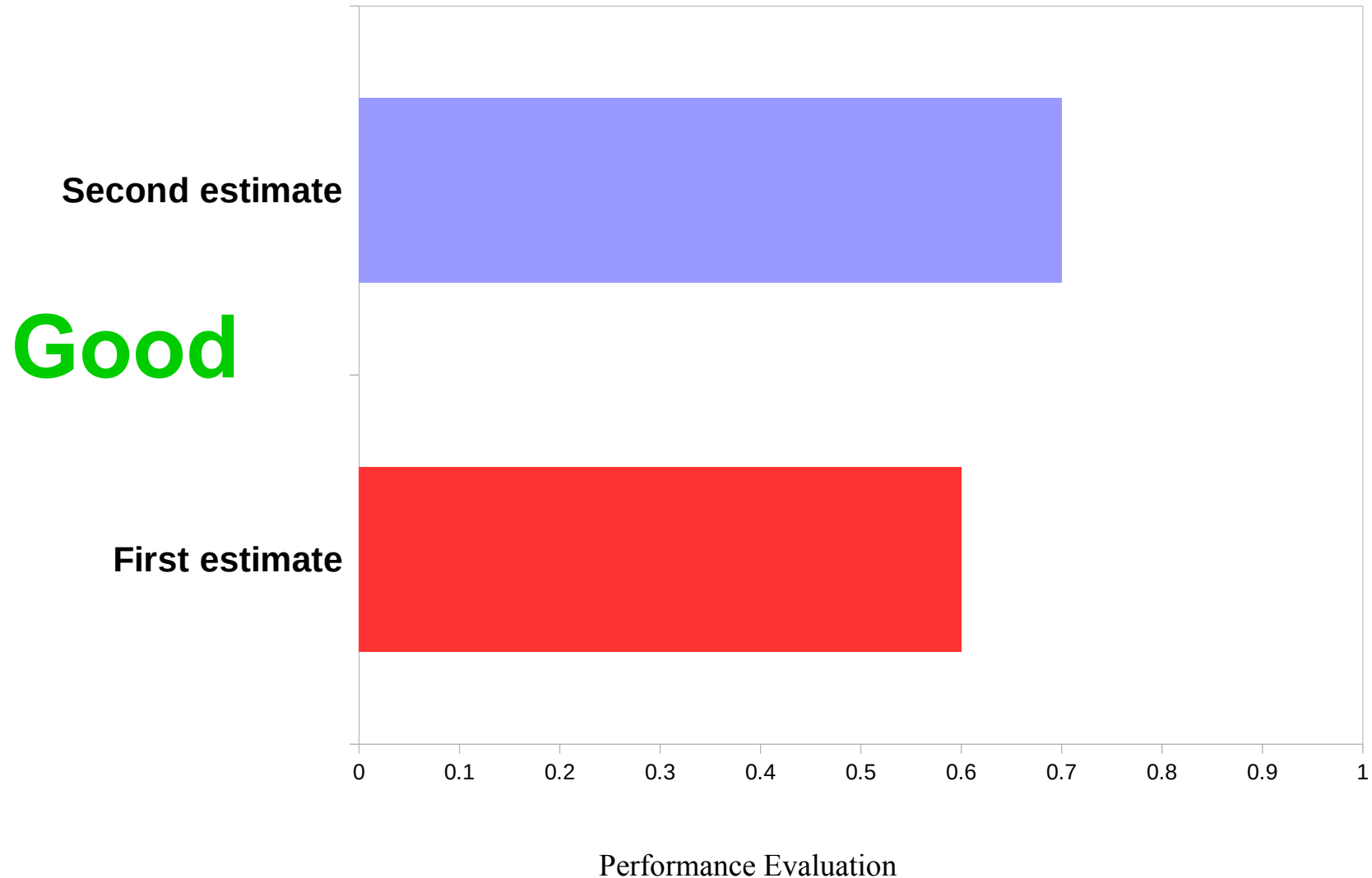
The Office for National Statistics (ONS) said gross domestic product (GDP) expanded by 0.7 per cent in the fourth quarter - an increase from the 0.6 per cent calculated on the watchdog's first look at the economy

<http://www.dailymail.co.uk/news/article-4248690/Economy-grew-0-7-final-three-months-2016.html>

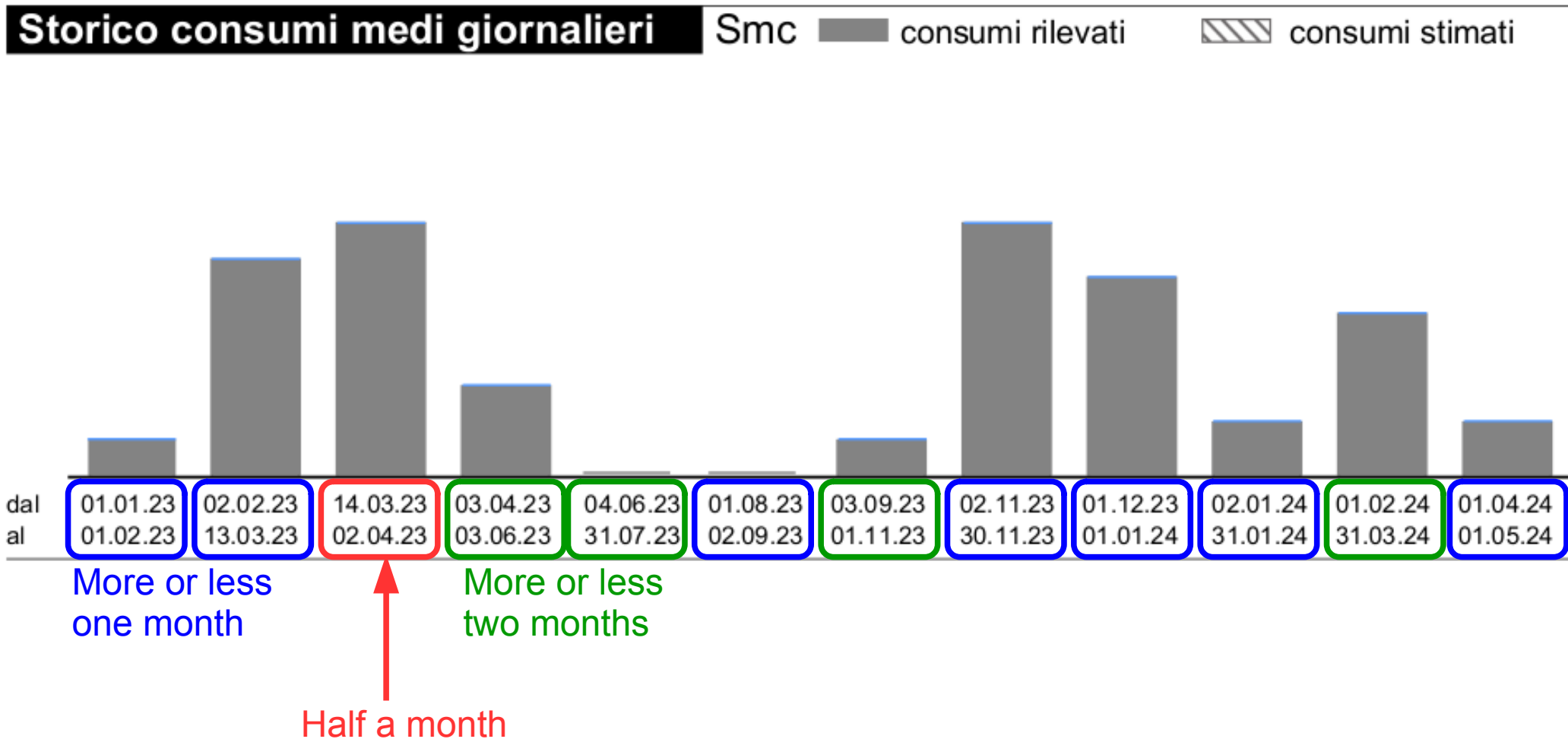
Do not misuse axis origins

a change from 0.6% to 0.7% with proper scale

2916 Q4 growth upgraded

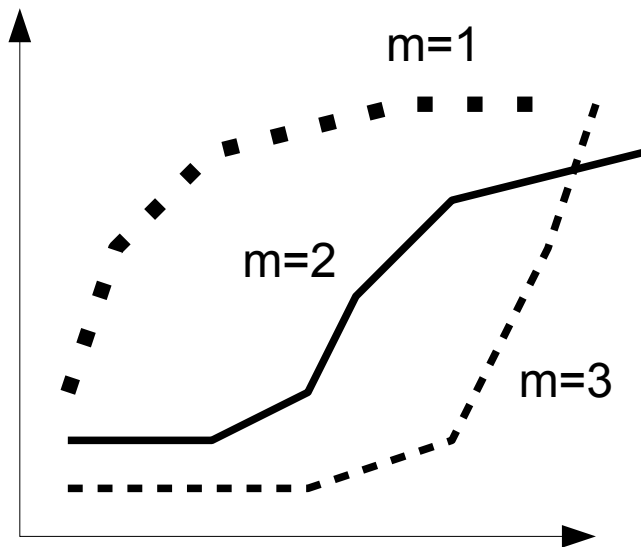


Do not misuse axis origins: Hera Edition

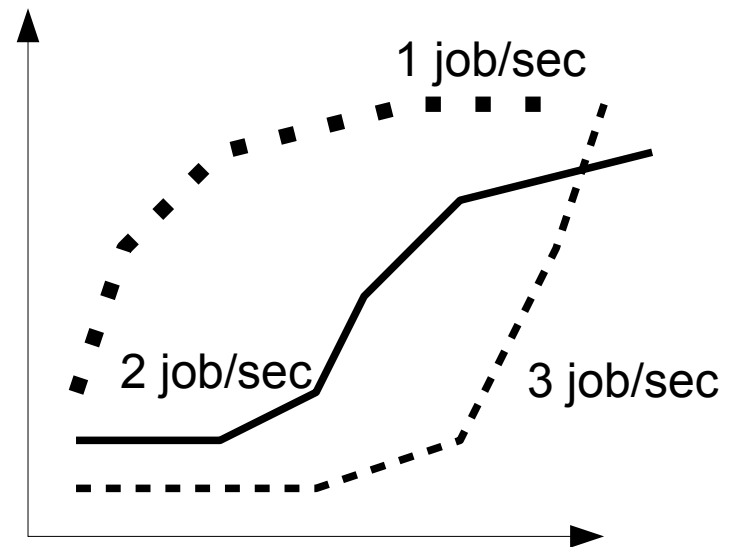


Prefer names to symbols

Bad

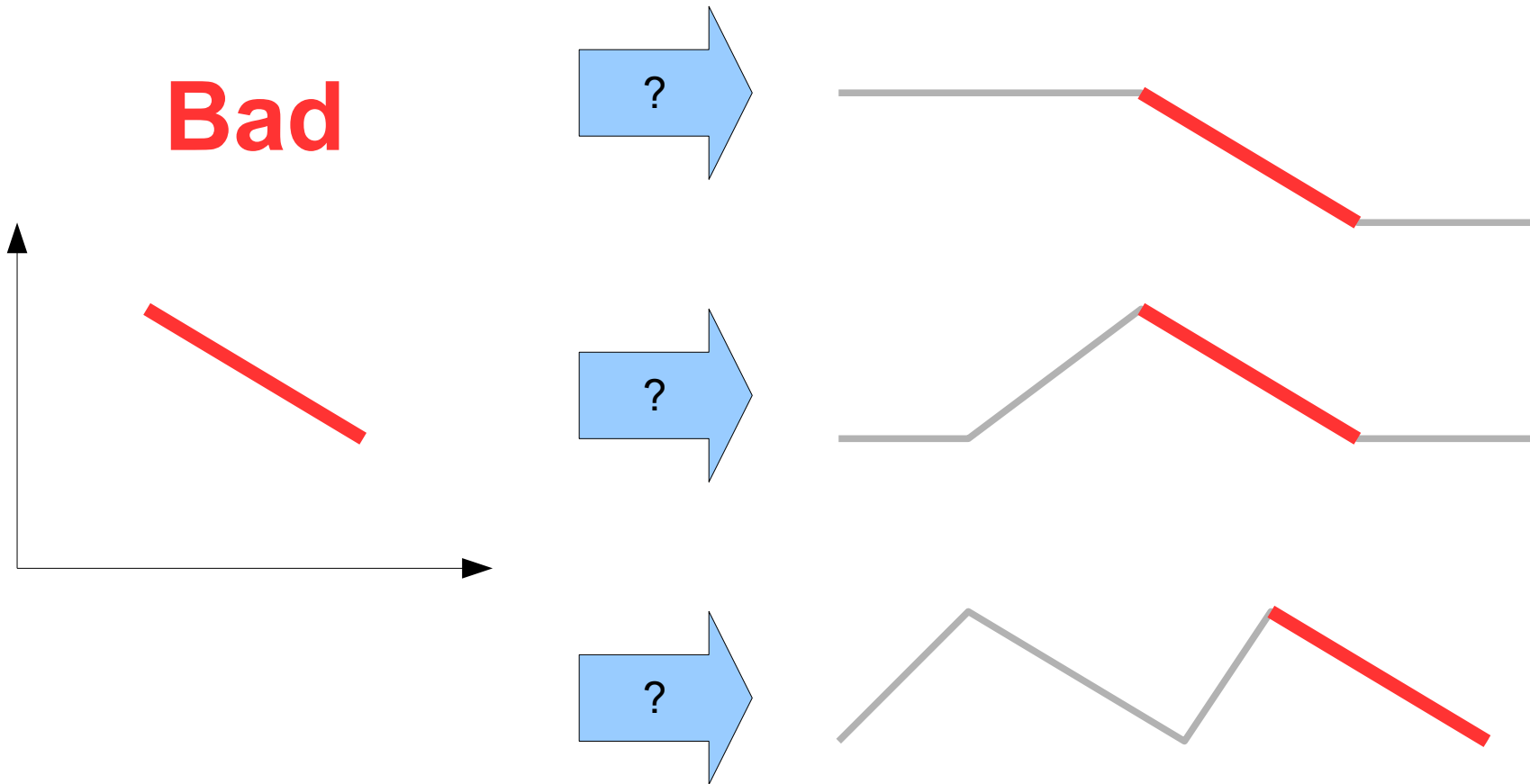


Good

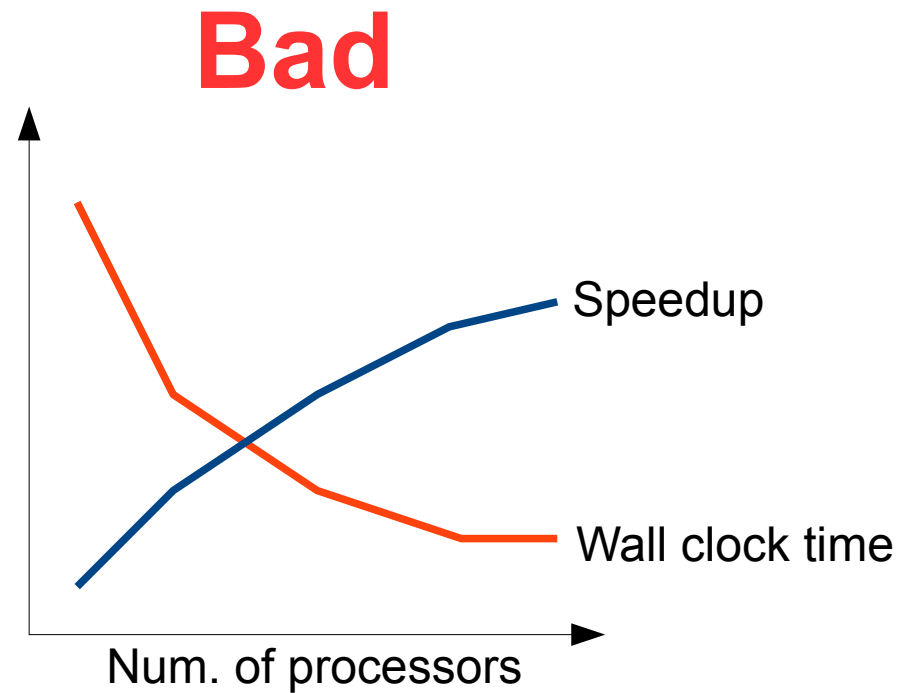


Show all data

Bad

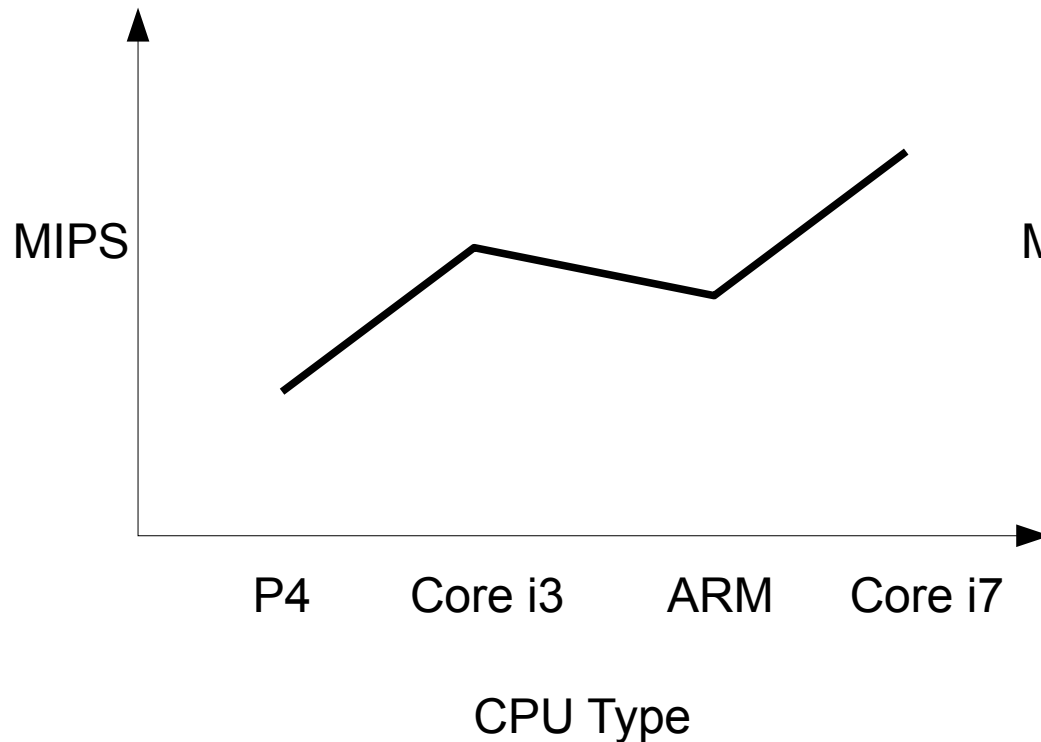


Do not overlap related data

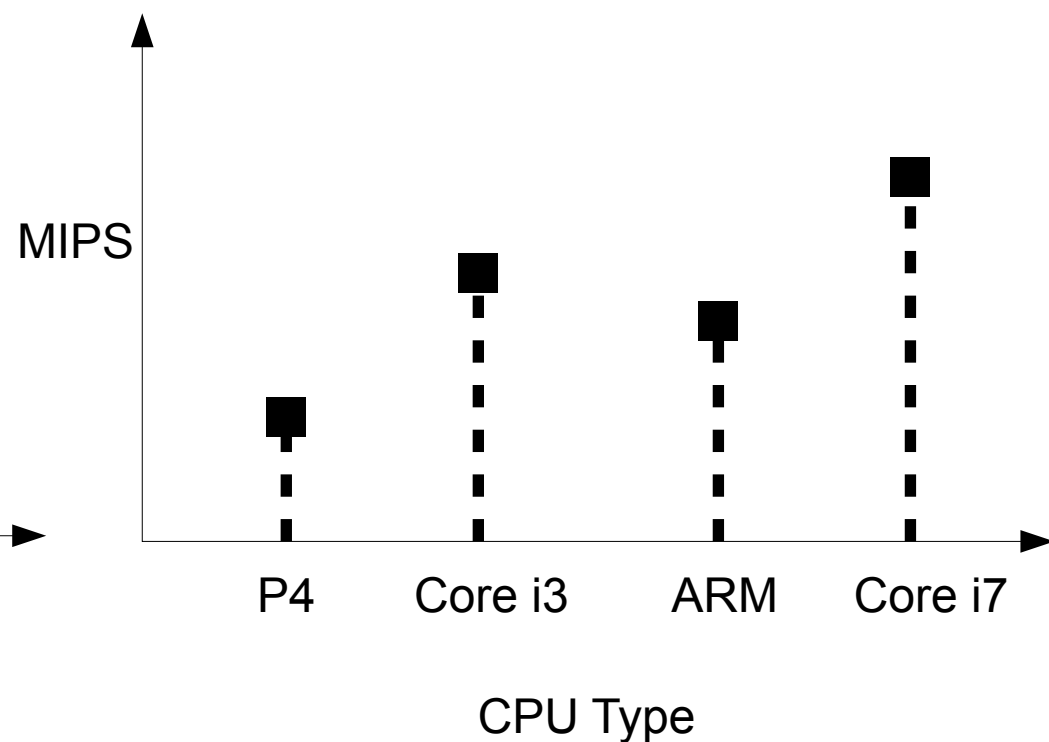


Do not connect points when intermediate values can not exist

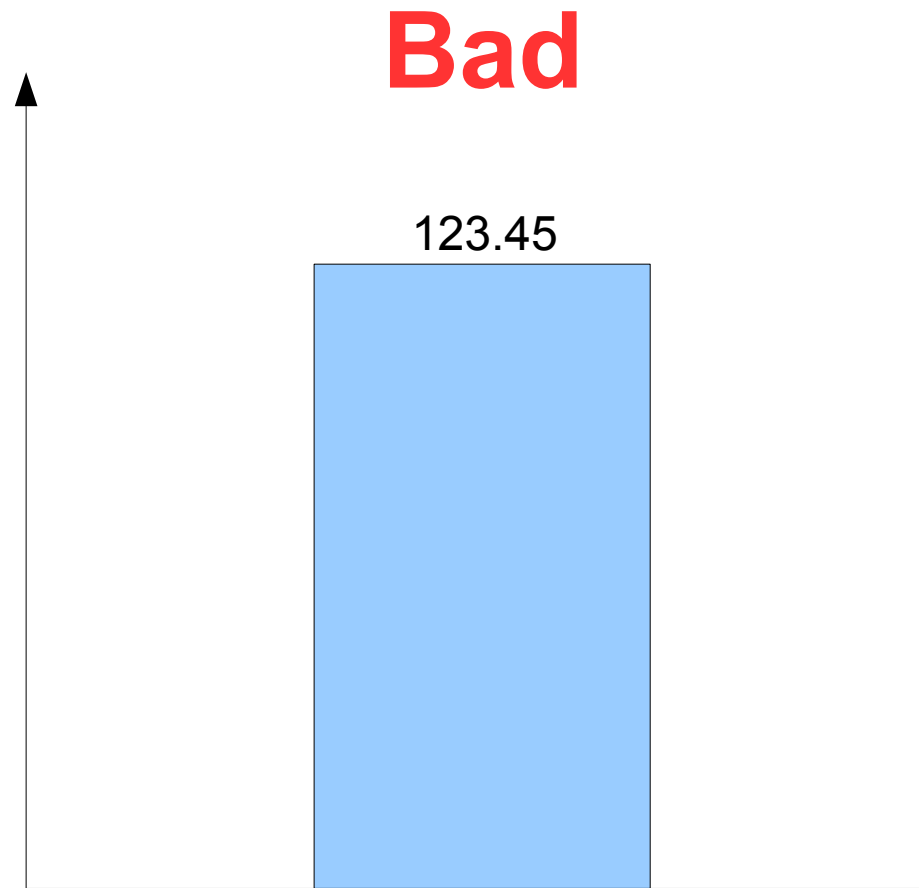
Bad



Better



Do not use graphs when words are enough



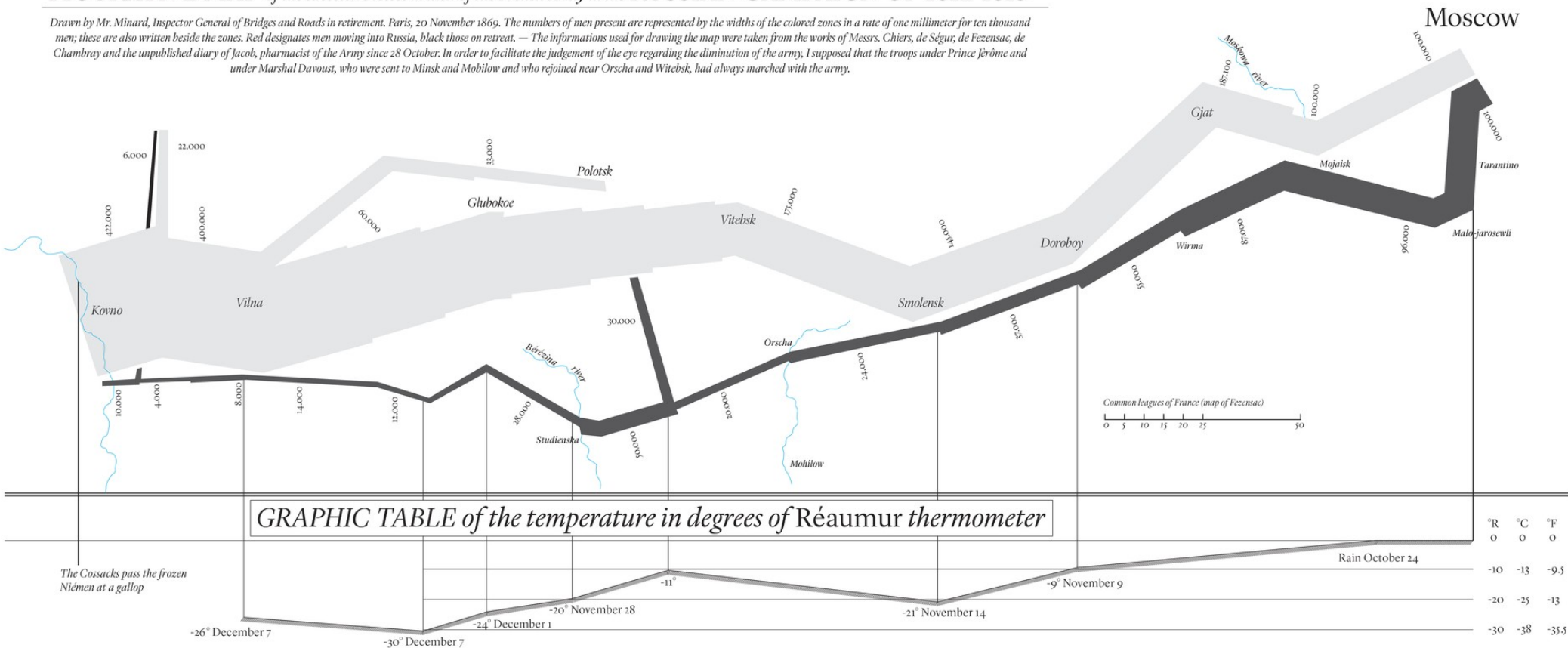
Other suggestions

- Each figure and table must have a **numbered caption**
 - Ideally, the caption should contain enough information to understand the figure without reading the main text
- All figures and tables must be **referenced** and **described** in the main text of your document

“The best statistical graphic ever drawn” (Edward Tufte)

FIGURATIVE MAP of the successive losses in men of the French Army in the RUSSIAN CAMPAIGN OF 1812-1813

Drawn by Mr. Minard, Inspector General of Bridges and Roads in retirement, Paris, 20 November 1869. The numbers of men present are represented by the widths of the colored zones in a rate of one millimeter for ten thousand men; these are also written beside the zones. Red designates men moving into Russia, black those on retreat. — The informations used for drawing the map were taken from the works of Messrs. Chiers, de Ségur, de Fezensac, de Chambray and the unpublished diary of Jacob, pharmacist of the Army since 28 October. In order to facilitate the judgement of the eye regarding the diminution of the army, I supposed that the troops under Prince Jérôme and under Marshal Davoust, who were sent to Minsk and Mobilow and who rejoined near Orscha and Witebsk, had always marched with the army.



Modern redrawing of [Charles Minard](#)'s map of Napoleon's disastrous Russian campaign of 1812. The graphic is notable for its representation in two dimensions of six types of data: the number of Napoleon's troops; distance; temperature; the latitude and longitude; direction of travel; and location relative to specific dates (Image and description from Wikipedia)